—----------Library.hs

```
module Library where
import PdePreludat


-- PERSONAS:
bombon = Persona {nombre = "bombon" , nivelDeResistencia = 55,habilidades = ["escuchar canciones", "golpes fuertes"],amigos = ["senorCerdo","silico"]}
burbuja = Persona {nombre = "burbuja", nivelDeResistencia = 30, habilidades = ["velocidad","burbujas"], amigos = ["senoritaBelo"]}
bellota = Persona {nombre = "bellota", nivelDeResistencia = 75, habilidades = ["velocidad","super fuerza"], amigos = []}
senoritaBelo = Persona {nombre = "senoritaBelo", nivelDeResistencia = 10, habilidades = ["no mostrar la cara "], amigos = ["burbuja"]}
senorCerdo = Persona {nombre = "senorCerdo", nivelDeResistencia = 0, habilidades = [], amigos = ["bellota"]}
silico = Persona {nombre = "silico", nivelDeResistencia = 29, habilidades = ["tolerar numeros impares"], amigos = ["bombon","burbuja"]}

--  AMENAZAS
mojoJojo = Amenaza { proposito ="destruir a las chicas super poderosas ", nivelDePoder = 70, debilidades= ["velocidad","super fuerza"],
efecto = rumorDeAtaque mojoJojo}
princesa = Amenaza{ proposito ="ser la unica chica super poderosa", nivelDePoder = 95, debilidades= ["burbujas","golpes fuertes"], efecto = todoIgual}
bandaGangrena = Amenaza { proposito ="esparcir caos, hacer flojos y peleen entre ellos", nivelDePoder = 49, debilidades= ["escuchar canciones","super fuerza", "kryptonita"],
efecto = efectoGangrena}

-- CIUDADES
saltadilla = Ciudad{nombreCiudad = "Saltadilla", cantidadHabitantes = 21}
stardewValley = Ciudad {nombreCiudad = "Stardew Valley", cantidadHabitantes = 27}
puebloPaleta = Ciudad {nombreCiudad = "Pueblo paleta", cantidadHabitantes = 10}

-- ESTRUCTURAS
data Persona = Persona {
    nombre :: String,
    nivelDeResistencia :: Number,
    habilidades :: [String],
    amigos :: [String]
} deriving (Show)

data Amenaza = Amenaza {
    proposito :: String,
    nivelDePoder :: Number,
    debilidades :: [String],
    efecto :: EfectoVillano
```

```
} deriving (Show)

data Ciudad = Ciudad {
   nombreCiudad :: String,
   cantidadHabitantes :: Number
} deriving (Show)

data Capitulo = Capitulo {
   villano :: Amenaza,
   chica :: Persona,
   ciudadAlQueAtacan :: Ciudad,
   alimentos :: [Alimento]
} deriving (Show)

-- 1RA ENTREGA

--1 !!
danioPotencialDe :: Amenaza -> Number
danioPotencialDe amenaza = (nivelDePoder amenaza -).(3*).length.debilidades $ amenaza

--2 !!
puedeAtacarCiudad :: Amenaza -> Ciudad -> Bool
puedeAtacarCiudad amenaza = (danioPotencialDe amenaza >).(2*).cantidadHabitantes

--3 !!
puedeVencerAmenaza :: Amenaza -> Persona -> Bool
puedeVencerAmenaza amenaza persona
   | even.length.proposito $ amenaza = resistenciaMayorA.(/2).danioPotencialDe $ amenaza
   | otherwise = resistenciaMayorA.danioPotencialDe $ amenaza
   where resistenciaMayorA = (nivelDeResistencia persona >)

--4 !!
noEsDebilAKriptonita ::  Amenaza -> Bool
noEsDebilAKriptonita =  not . elem "kriptonita" . debilidades

tieneCantidadParDeDebilidades :: Amenaza -> Bool
tieneCantidadParDeDebilidades = even . length . debilidades

esNivelAlto :: Amenaza -> Bool
esNivelAlto amenaza = (((tieneCantidadParDeDebilidades amenaza &&).(>
50).danioPotencialDe $ amenaza) &&).noEsDebilAKriptonita $ amenaza


-- FIN 1RA ENTREGA
```

```
--2DA ENTREGA

-- ENTRADA EN CALOR

--1 !!

esDebilFrenteA :: Amenaza -> String -> Bool
esDebilFrenteA amenaza habilidad = elem habilidad debilidadesAmenaza
   where debilidadesAmenaza = debilidades amenaza

esAmenazaInmune :: Amenaza -> Persona -> Bool
esAmenazaInmune amenaza persona = not.any (esDebilFrenteA amenaza) $
listaHabilidades
   where listaHabilidades = habilidades persona


--2 !!
compararPoder :: Amenaza -> Amenaza -> Amenaza
compararPoder amenaza1 amenaza2
   | (nivelDePoder amenaza1 >).nivelDePoder $ amenaza2 = amenaza1
   | otherwise = amenaza2

amenazaPreponderante :: [Amenaza] -> Amenaza
amenazaPreponderante = foldl1 compararPoder

--3 !!
puedenVencerAmenaza :: [Persona] -> Amenaza -> [String]
puedenVencerAmenaza [] _ = []
puedenVencerAmenaza personas amenaza = (map nombre ).filtrarQuienesDerrotan
amenaza $ personas

filtrarQuienesDerrotan :: Amenaza -> [Persona] -> [Persona]
filtrarQuienesDerrotan amenaza = filter (puedeVencerAmenaza amenaza)

--4 !!

nombreEmpiezaConBYMasDeUnaHabilidad :: [Persona] -> [Persona]
nombreEmpiezaConBYMasDeUnaHabilidad = filter evaluarNombreYHabilidades

evaluarNombreYHabilidades :: Persona -> Bool
evaluarNombreYHabilidades persona = (nombreEmpiezaConB persona
&&).masDeUnaHabilidad $ listaHabilidades
   where listaHabilidades = habilidades persona
```

```
nombreEmpiezaConB :: Persona -> Bool
nombreEmpiezaConB = (=='b').head.nombre

masDeUnaHabilidad :: [String] -> Bool
masDeUnaHabilidad = (>1).length

-- FIN ENTRADA EN CALOR




-- YENDO AL NUTRICIONISTA

type Alimento = Persona -> Persona

modificarResistencia :: Persona -> Number -> Persona
modificarResistencia persona valor = persona{nivelDeResistencia = resistenciaPersona +
valor}
   where resistenciaPersona = nivelDeResistencia persona

--1 !!
hacerAmigos :: Persona -> [String] -> Persona
hacerAmigos persona juntada = persona{amigos = (listaAmigos ++).filtrarJuntada
listaAmigos $ juntada}
   where listaAmigos = amigos persona

filtrarJuntada :: [String] -> [String] -> [String]
filtrarJuntada listaAmigos juntada = filter (\x -> x `notElem` listaAmigos) juntada

beberCerveza :: [String] -> Alimento
beberCerveza juntada persona = hacerAmigos persona juntada

beberSustanciaX :: Alimento
beberSustanciaX persona = modificarResistencia persona resistenciaNegativa
   where resistenciaNegativa = (*(-1)).nivelDeResistencia $ persona

--2 !!

beberSaborizante :: String -> Alimento
beberSaborizante sabor persona = modificarResistencia persona letrasRestadas
   where letrasRestadas = (*(-1)).length $ sabor
```

```
beberFernet :: Alimento
beberFernet persona
    | not.(elem "Chef de Asados") $ listaHabilidades = persona {habilidades =
listaHabilidades ++ ["Chef de Asados"]}
    | otherwise = persona
   where listaHabilidades = habilidades persona

--3 !!

beberGatorei :: Alimento
beberGatorei persona = modificarResistencia persona.(5*).length $ listaAmigos
   where listaAmigos = amigos persona

beberVodka :: Number -> Alimento
beberVodka shots persona = persona{nombre = reverse.drop(min shots
letrasNombre).reverse.nombre $ persona}
   where letrasNombre = length.nombre $ persona

--4 !!

beberCarameloLiquido :: Alimento
beberCarameloLiquido persona = modificarResistencia persona (-10)

beberCocucha :: Alimento
beberCocucha persona =  quitarPrimeraHabilidad listaHabilidades persona
   where listaHabilidades = habilidades persona

quitarPrimeraHabilidad :: [String] -> Persona -> Persona
quitarPrimeraHabilidad [] persona = persona
quitarPrimeraHabilidad listaHabilidades persona = persona{habilidades = drop 1
listaHabilidades}

-- FIN YENDO AL NUTRICIONISTA




-- MI VILLANO FAVORITO

type EfectoVillano = Ciudad -> Ciudad
```

```
atacarCiudad :: Amenaza -> EfectoVillano
atacarCiudad amenaza ciudad
   | puedeAtacarCiudad amenaza ciudad = efectoAmenaza.rumorDeAtaque amenaza $
ciudad
   | otherwise = rumorDeAtaque amenaza ciudad
   where efectoAmenaza = efecto amenaza

rumorDeAtaque :: Amenaza -> EfectoVillano
rumorDeAtaque amenaza ciudad = ciudad{cantidadHabitantes = (max
0).(cantidadHabitantes ciudad -).(div (danioPotencialDe amenaza)) $ 10}

--efecto princesa

todoIgual :: EfectoVillano
todoIgual = id

--efecto gangrena

efectoGangrena :: EfectoVillano
efectoGangrena ciudad = ciudad{nombreCiudad = "Gangrena City", cantidadHabitantes =
(2* ).cantidadHabitantes $ ciudad}


-- FIN MI VILLANO FAVORITO




-- DARLE PLAY

cap1 = Capitulo{villano = mojoJojo, chica = bombon, ciudadAlQueAtacan = saltadilla,
alimentos = [beberSustanciaX]}
cap2 = Capitulo{villano = princesa, chica = burbuja, ciudadAlQueAtacan = saltadilla,
alimentos=[beberSustanciaX]}
cap3 = Capitulo{villano= princesa,
chica=burbuja,ciudadAlQueAtacan=stardewValley,alimentos=[beberFernet]}
cap4 = Capitulo{villano = bandaGangrena, chica = bellota , ciudadAlQueAtacan =
puebloPaleta , alimentos = [beberSustanciaX, beberCocucha]}

darlePlay :: Ciudad -> Capitulo -> Ciudad
darlePlay ciudad capitulo
  | capituloAfectaCiudad ciudad capitulo = atacarCiudad (villano capitulo) ciudad
  |  otherwise = ciudad
```

```
capituloAfectaCiudad :: Ciudad -> Capitulo -> Bool
capituloAfectaCiudad ciudad capitulo = (((ciudadDelCapitulo ==).nombreCiudad $ ciudad)
&&).noPuedeDefenderCiudad $ capitulo
   where ciudadDelCapitulo = nombreCiudad.ciudadAlQueAtacan $ capitulo

noPuedeDefenderCiudad :: Capitulo -> Bool
noPuedeDefenderCiudad capitulo = not $ puedeVencerAmenaza (villano capitulo)
(chicaModificada)
   where chicaModificada = consumirTodosLosAlimentos capitulo

consumirTodosLosAlimentos :: Capitulo -> Persona
consumirTodosLosAlimentos capitulo = foldr ($) (chica capitulo) (alimentos capitulo)

maraton :: Ciudad -> [Capitulo] -> Ciudad
maraton ciudad = foldl darlePlay ciudad

temporada = [cap1, cap2, cap3, cap4]

-- FIN DARLE PLAY

—----------------Spec.hs

module Spec where
import PdePreludat
import Library
import Test.Hspec

correrTests :: IO ()
correrTests = hspec $ do

  describe "1er Entrega - Test Punto 1" $ do
     it "danioPotencialDe mojoJojo es 64" $ do
       danioPotencialDe mojoJojo `shouldBe` 64

     it "danioPotencialDe de princesa es 89" $ do
        danioPotencialDe princesa `shouldBe` 89

     it "danioPotencialDe de  bandaGangrena es 40" $ do
        danioPotencialDe bandaGangrena `shouldBe` 40

  describe "1er Entrega - Test Punto 2" $ do
     it "mojoJojo puede atacar Saltadilla" $ do
       saltadilla `shouldSatisfy` puedeAtacarCiudad mojoJojo

     it "princesa puede atacar Saltadilla" $ do
       saltadilla `shouldSatisfy` puedeAtacarCiudad princesa
```

```
    it "bandaGangrena no puede atacar Saltadilla" $ do
      saltadilla `shouldNotSatisfy` puedeAtacarCiudad bandaGangrena

  describe "1er Entrega - Test Punto 3" $ do
    it "bombon puede vencer a mojoJojo" $ do
      bombon `shouldSatisfy` puedeVencerAmenaza mojoJojo

    it "burbuja no puede vencer a princesa" $ do
      burbuja `shouldNotSatisfy` puedeVencerAmenaza princesa

  describe "1er Entrega - Test Punto 4" $ do
    it "princesa es una amenaza de nivel alto" $ do
      princesa `shouldSatisfy` esNivelAlto

    it "bandaGangrena no es una amenaza de nivel alto" $ do
      bandaGangrena `shouldNotSatisfy` esNivelAlto




  describe "Yendo al Nutricionista - Test Punto 1" $ do
    it "bombon se queda sin resistencia luego de consumir la sustancia X" $ do
      nivelDeResistencia (beberSustanciaX bombon) `shouldBe` 0

    it "burbuja se queda sin resistencia luego de consumir la sustancia X" $ do
      nivelDeResistencia (beberSustanciaX bombon) `shouldBe` 0

    it "bellota se queda sin resistencia luego de consumir la sustancia X" $ do
      nivelDeResistencia (beberSustanciaX bombon) `shouldBe` 0

    it "burbuja toma una cerveza con silico y queda con 2 amigos" $ do
      amigos (beberCerveza ["silico"] burbuja) `shouldBe` ["senoritaBelo", "silico"]

    it "burbuja toma una cerveza con la senoritaBelo y queda con un amigo" $ do
      amigos (beberCerveza ["senoritaBelo"] burbuja)  `shouldBe` (amigos burbuja)

  describe "Yendo al Nutricionista- Test Punto 2" $ do

    it "bellota se toma un fernet y tiene su nueva habilidad de Chef de Asados" $ do
      habilidades (beberFernet bellota) `shouldBe` ["velocidad","super fuerza","Chef de
Asados"]

  describe "Yendo al Nutricionista - Test Punto 3" $ do
```

```
    it "bellota luego de tomar gatorei queda igual" $ do
      nivelDeResistencia (beberGatorei bellota) `shouldBe` (nivelDeResistencia bellota)

    it "bombon luego de tomar gatorei queda con 65 puntos de resistencia" $ do
      nivelDeResistencia (beberGatorei bombon) `shouldBe` 65

    it "bombon toma 3 shots de vodka y pasa a llamarse bom" $ do
      nombre (beberVodka 3 bombon) `shouldBe` "bom"

  describe "Yendo al Nutricionista - Test Punto 4" $ do
    it "burbuja debe tener 20 de resistencia luego de consumir el caramelo líquido" $ do
      nivelDeResistencia (beberCarameloLiquido burbuja) `shouldBe` 20

    it "senoritaBelo debe tener 0 de resistencia luego de consumir caramelo líquido" $ do
      nivelDeResistencia (beberCarameloLiquido senoritaBelo) `shouldBe` 0

    it "senorCerdo le entra a la cocucha y queda igual" $ do
      habilidades (beberCocucha senorCerdo) `shouldBe` habilidades senorCerdo




  describe "Mi Villano Favorito - Tests" $ do
    it "mojoJojo ataca saltadilla, su poblacion final es 9" $ do
      cantidadHabitantes (atacarCiudad mojoJojo saltadilla)  `shouldBe` 9

    it "princesa ataca saltadilla, su poblacion final es 13" $ do
      cantidadHabitantes (atacarCiudad princesa saltadilla)  `shouldBe` 13

    it "bandaGangrena ataca saltadilla 1 vez, su poblacion final es 17 " $ do
      cantidadHabitantes (atacarCiudad bandaGangrena saltadilla)  `shouldBe` 17

    it "bandaGangrena ataca saltadilla 2 veces consecutivas, su poblacion final es 26" $ do
      cantidadHabitantes (atacarCiudad bandaGangrena (atacarCiudad bandaGangrena
saltadilla)) `shouldBe` 26

    it "bandaGangrena ataca saltadilla 2 veces consecutivas, pasa a llamarse Gangrena
City" $ do
      nombreCiudad (atacarCiudad bandaGangrena (atacarCiudad bandaGangrena
saltadilla)) `shouldBe` "Gangrena City"
```

```
describe "Darle Play" $ do
    it "Capitulo 1 afecta a Saltadilla" $ do
      cantidadHabitantes (darlePlay saltadilla cap1) `shouldBe` 9

    it "Capitulo 1 NO afecta a Pueblo Paleta" $ do
      cantidadHabitantes (darlePlay puebloPaleta cap1) `shouldBe` 10

    it "Capitulo 2 afecta a Saltadilla" $ do
      cantidadHabitantes (darlePlay saltadilla cap2) `shouldBe` 13

    it "Capitulo 2 NO afecta a Stardew Valley" $ do
      cantidadHabitantes (darlePlay  stardewValley cap2) `shouldBe` 27

    it "Capitulo 3 afecta a Stardew Valley" $ do
      cantidadHabitantes (darlePlay stardewValley cap3) `shouldBe` 19

    it "Capitulo 3 NO afecta a Pueblo Paleta" $ do
      cantidadHabitantes (darlePlay puebloPaleta cap3) `shouldBe` 10

    it "Capitulo 4 afecta a Pueblo Paleta" $ do
      cantidadHabitantes (darlePlay puebloPaleta cap4) `shouldBe` 12

    it "Capitulo 4 NO afecta a Saltadilla" $ do
      cantidadHabitantes (darlePlay saltadilla cap4) `shouldBe` 21



describe "Maraton" $ do

    it "La temporada deja a Saltadilla con 1 habitante" $ do
      cantidadHabitantes(maraton saltadilla temporada) `shouldBe` 1

    it "La temporada deja a Pueblo Paleta con 12 habitantes" $ do
      cantidadHabitantes(maraton puebloPaleta temporada) `shouldBe` 12
```